

Classes & Methods

The Behavior of Objects

A Technical Guide to Class Member Functions

DEFINITION • SCOPE • OPTIMISATION

1. Logic within Objects

A **Method** is simply a function that belongs to a class. While attributes represent the data of an object, methods represent its actions or behaviors.

Key Rule:

Methods have special access: they can see and modify all attributes inside their own class, even if those attributes are private.

The Mechanics:

- Methods are defined inside the class scope.
- They are called using the **Dot Operator** (`object.method()`).
- They allow objects to interact with their own data.

2. Inline Method Definitions

The simplest way to create a method is to define the entire logic directly inside the `class { ... }` block.

```
class CodeHive {
public:
    string courseName;

    // Method defined INSIDE the class
    void showDetails() {
        cout << "Course: " << courseName;
    }
};
```

When to use: This is perfect for small functions with only a few lines of code. C++ often treats these as "inline" automatically for performance.

3. Scope Resolution (::)

In professional projects, class bodies are kept clean by only declaring methods inside and defining them outside.

```
class CodeHive {  
public:  
    void showDetails(); // Declaration only  
};  
  
// Definition outside using the Scope Resolution Operator  
void CodeHive::showDetails() {  
    cout << "Details displayed."  
}
```

The `::` tells the compiler: "This function belongs to the CodeHive class, don't confuse it with a normal global function!"

4. Parameterised Methods

Methods become powerful when they accept external data through parameters. This allows an object to react to outside information.

```
class CodeHive {  
public:  
    void greet(string studentName) {  
        cout << "Welcome, " << studentName;  
    }  
};  
  
int main() {  
    CodeHive myHive;  
    myHive.greet("Avni"); // Passing the argument  
}
```

The method acts as a bridge between the object's internal state and external user input.

5. Output and Calculation

Methods aren't just for printing; they can perform complex calculations and **return** the result to the caller.

```
class Calculator {  
public:  
    int multiply(int a, int b) {  
        return a * b;  
    }  
};  
  
int main() {  
    Calculator calc;  
    int product = calc.multiply(5, 4); // Product is 20  
}
```

Concept: Returning values allows you to use an object as a "Logic Engine" for your program.

6. Read-Only Methods (const)

To write safe, high-quality code, use the `const` keyword for methods that **should not** change any data variables.

```
class User {  
    string username;  
public:  
    // This method is guaranteed NOT to modify username  
    void printName() const {  
        cout << username;  
        // username = "new"; // ❌ Compiler Error!  
    }  
};
```

Using `const` prevents accidental bugs and allows you to use the method with "const objects."

7. Micro-Optimisation

Small methods can be marked as `inline`. This tells the compiler to replace the function call with the actual code to save time.

```
class Point {  
public:  
    inline void fastGreet() {  
        cout << "Hi!";  
    }  
};
```

Note: Modern compilers usually "inline" small functions automatically, but the `inline` keyword serves as a helpful hint for performance-critical loops.

8. Private vs. Public Methods

Not all methods should be usable by everyone. Some are "Helper functions" used only internally by the class.

```
class SecureDrive {  
private:  
    void decryptData() { /* Hidden Logic */ }  
  
public:  
    void openFiles() {  
        decryptData(); // Only the class can call this  
        cout << "Files ready.";  
    }  
};
```

This follows the principle of **Encapsulation**—keeping complexity hidden behind simple public methods.

9. Architectural Guidelines

1. Clear Naming: Use verbs for methods (e.g., `calculateTotal()`, `sendEmail()`). Attributes should be nouns.

2. Single Responsibility: One method should solve exactly one problem. Don't make `processOrder()` also handle `printInvoice()`.

3. Keep Class Bodies Slim: Use the `::` operator outside the class for logic longer than 5 lines to keep your header files readable.

10. Comprehensive Summary

- **Inside:** Great for speed and small tasks.
- **Outside (::):** Standard for large, professional projects.
- **Parameters:** Input data for flexibility.
- **Return:** Send results back to the program.
- **Const:** Protection against accidental data modification.
- **Dot (.):** The gateway to calling behaviors.

```
// FINAL CHALLENGE:  
// Write a class 'Wallet' with a private attribute 'money'.  
// Create a public method 'addFunds(int amt)'  
// and a const method 'showBalance()'.
```

THEORY & PRACTICE: COMPLETE